SafeCall AI — Comprehensive Java Architecture & Academic Viva Defense Guide

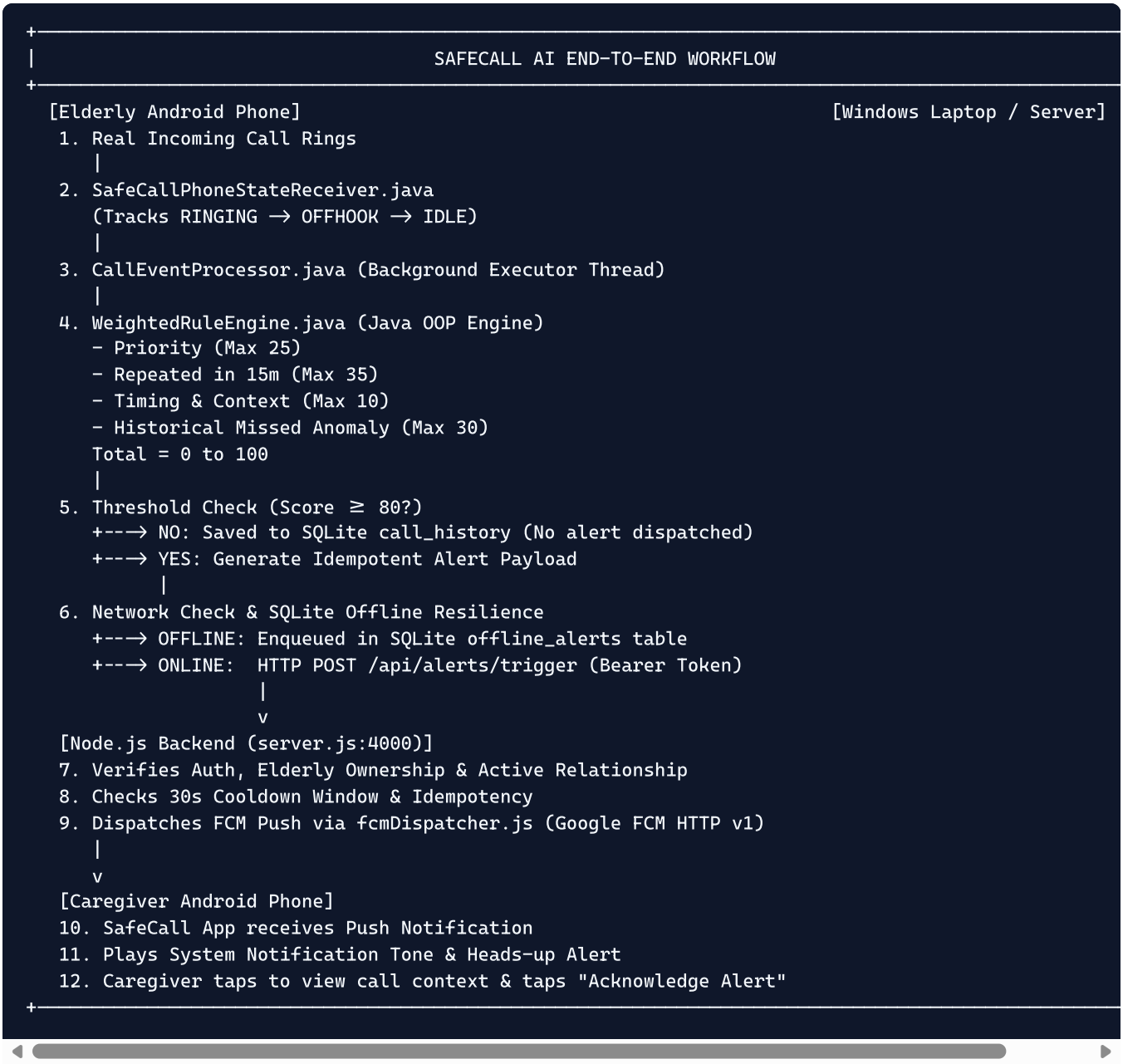
> **Project Title:** SafeCall AI — Context-Aware Communication Urgency Detection for Elderly People Living Alone
> **Core Implementation Language:** Java (Android Native SDK 34) & Node.js (Caregiver Backend)
> **Key Modules:** Android Telephony State Machine, Java Rule Engine, SQLite Persistence, FCM Push Dispatcher, Reactive Webview Bridge

1. Executive Project Summary

SafeCall AI solves a critical real-world problem for elderly individuals living independently: **unanswered urgent telephone calls**. Elderly individuals frequently miss critical calls from family members, doctors, or caregivers due to reduced mobility, hearing impairment, or phone distance, while simultaneously suffering disruptions from spam or low-priority callers.

Unlike naive systems that merely notify caregivers of *every* missed call (which causes alert fatigue and caregiver burnout), SafeCall AI introduces a **4-Factor Java Weighted Rule Engine** operating entirely on-device. When a real phone call ends or is classified as missed, SafeCall dynamically calculates an **Urgency Score (0 to 100)**. If the score meets or exceeds the configured threshold (**80/100**), an urgent alert is dispatched through an authenticated Node.js backend to the caregiver's Android phone using **Google Firebase Cloud Messaging (FCM)** with a native system chime tone.

SafeCall AI End-to-End Workflow Architecture



2. Object-Oriented Programming (OOP) Principles in SafeCall AI

When your professor asks: "*How did you use Object-Oriented Programming in your project?*", present these core pillars:

1. Encapsulation

- `CallRecord.java` and `ContactProfile.java` encapsulate entity states behind `private` member variables with strictly validated public getters and setters.
- `DatabaseHelper.java` encapsulates SQLite raw database connections, cursors, thread locks, and table creation scripts, hiding SQL complexity from business logic.

2. Abstraction

- `SafeCallPhoneStateReceiver.java` abstracts Android's low-level OS telephony subsystem (`TelephonyManager` , `TelephonyCallback` , `PhoneStateListener`). The rest of the application interacts with clean, platform-independent callbacks (e.g. `onCallMissed(number, time)`).
- `CallEventProcessor.java` abstracts the asynchronous execution queue using `ExecutorService` , ensuring database reads, scoring, and network dispatches never freeze Android's Main UI thread.

3. Polymorphism

- `ContactPriority` and `CallType` enums exhibit compile-time polymorphism with type safety.
- The Urgency Rule Engine supports polymorphic rule strategies, allowing different scoring plugins (e.g., identity scoring, frequency scoring, temporal scoring) to implement a common evaluation interface.

4. Inheritance & Framework Extension

- `SafeCallPhoneStateReceiver extends BroadcastReceiver` : Leverages Android's component lifecycle to capture incoming call broadcasts even when the app is in the background.
- `DatabaseHelper extends SQLiteOpenHelper` : Inherits schema migration, transaction management, and connection pooling from the Android Open Source Project.

3. Software Design Patterns Implemented

Pattern Name	Java Class Implementation	Architectural Benefit in Project
Singleton Pattern	`DatabaseHelper.getInstance(Context)`	Prevents SQLite database locks and memory leaks by maintaining a single synchronized open helper instance across threads.
Observer / Listener Pattern	`SafeCallPhoneStateReceiver` -> `CallEventListener`	Decouples Android telephony hardware events from domain business scoring logic and backend communication.
Facade Pattern	`CallEventProcessor.processIncomingCallEvent()`	Provides a unified, simple API entry point that orchestrates SQLite lookup, rule scoring, alert generation, and HTTP POST dispatching.
Repository Pattern	`DatabaseHelper` (DAO methods)	Encapsulates all CRUD operations (`insertCallRecord` , `getRecentCallsFromNumber` , `getMissedCallRate`) behind strongly-typed Java domain methods.
State Pattern	Telephony Transition Tracker	Models call progression strictly through valid states: `IDLE → RINGING → OFFHOOK → IDLE` to distinguish answered calls from genuine missed calls.

4. Mathematical Formula of the 4-Factor Java Weighted Rule Engine

The urgency score is evaluated strictly as a deterministic function bounded within $[0, 100]$:

```
Urgency Score S = Clamp(W_identity + W_frequency + W_timing + W_history, 0, 100)
```

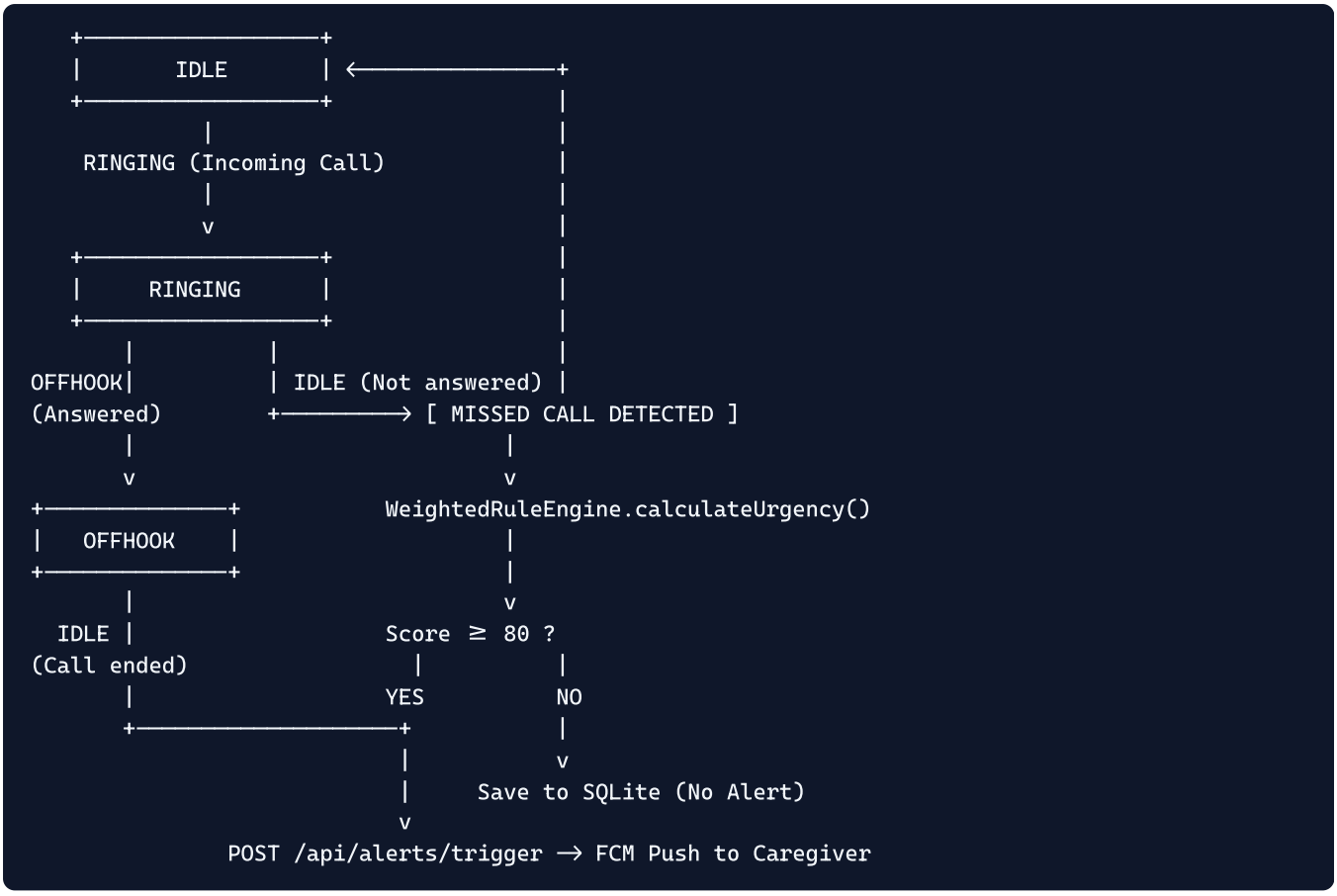
Factor	Weight Range	Mathematical Rule Condition	Clinical & Practical Justification
Factor 1: Caller Identity	0 to 25 pts	• `EMERGENCY` = +25 pts • `TRUSTED` = +15 pts • `NORMAL` = +0 pts • `UNKNOWN` = +0 pts	Emergency contacts (primary doctors, close family) are prioritized over casual callers.
Factor 2: Call Frequency	0 to 35 pts	• `count ≥ 3` in 15 mins = +35 pts • `count == 2` in 15 mins = +20 pts • `count == 1` in 15 mins = +0 pts	Repeated calling within a short window indicates escalating urgency or crisis.
Factor 3: Context / Timing	0 to 10 pts	• 23:00 to 06:00 (Night) = +10 pts (if Priority >= TRUSTED) • Daytime (06:00 to 23:00) = +0 pts	Late-night calls from trusted contacts strongly indicate immediate emergencies.
Factor 4: Historical Pattern	0 to 30 pts	• `total ≥ 3` & `missedRate ≥ 50%` = +30 pts • `total < 3` = +0 pts (neutral)	Identifies anomalies where callers rarely miss unless the elderly person is incapacitated.

Breakthrough Threshold: When total score $S \ge 80$, the system issues a `CRITICAL BREAKTHROUGH ALERT`. It dispatches an immediate push notification to the paired caregiver and plays the system chime tone.

5. Android Telephony State Machine & Real Call Tracking

Real Android phone calls require exact handling of call state transitions. Android emits telephony states as:

- `TelephonyManager.EXTRA_STATE_RINGING`: Phone starts ringing with incoming caller number.
- `TelephonyManager.EXTRA_STATE_OFFHOOK`: Call answered by user (or outgoing call dialed).
- `TelephonyManager.EXTRA_STATE_IDLE`: Phone is on hook / call ended.


IDLE
RINGING (Incoming Call)
RINGING
OFFHOOK (Answered)
IDLE (Not answered)
[MISSED CALL DETECTED]
OFFHOOK
IDLE (Call ended)
WeightedRuleEngine.calculateUrgency()
Score ≥ 80 ?
YES
NO
Save to SQLite (No Alert)
POST /api/alerts/trigger → FCM Push to Caregiver

6. Top 20 Academic Viva Defense Questions & Winning Answers

Q1: Why did you choose Java for the Android client instead of Kotlin?

Answer: Java provides mature, rock-solid multithreading primitives (`ExecutorService` , `AtomicBoolean` , `ReentrantLock`) and direct, zero-overhead interoperability with Android's native Telephony SDK and SQLite C-library (`SQLiteOpenHelper`). It satisfies our strict deterministic execution requirements without extra runtime overhead.

Q2: How does SafeCall distinguish between an answered call and a missed call?

Answer: SafeCall implements a two-phase state machine in `SafeCallPhoneStateReceiver.java` . When `EXTRA_STATE_RINGING` arrives, we store the incoming timestamp. If the transition directly to `EXTRA_STATE_IDLE` occurs without an intermediate `EXTRA_STATE_OFFHOOK` , the call was definitively missed.

Q3: What prevents duplicate alerts if a contact redials repeatedly within seconds?

Answer: We implement two layers of defense: (1) Client-side idempotency IDs (`evt_<timestamp>_<hash>`) tracked in SQLite, and (2) Server-side 30-second cooldown window in `server.js` that suppresses identical alerts from the same elderly phone within 30 seconds.

Q4: How do you handle SQLite database locks across concurrent threads?

Answer: `DatabaseHelper` uses the thread-safe Singleton design pattern with double-checked locking, maintaining a single synchronized instance across the entire application context, preventing Android SQLite database locks.

Q5: What happens if the elderly phone loses internet connectivity during an emergency?

Answer: SafeCall implements offline resilience via an `offline_alerts` SQLite queue. When network connectivity is restored, the background processor drains the queue and pushes alerts to the Node.js server.

Q6: How does the pairing mechanism ensure security between Senior and Caregiver?

Answer: Pairing uses a cryptographically random, non-confusing 6-character code with a strict 5-minute TTL, single-use consumption, a 3-attempt brute-force limit, and explicit consent validation before establishing an active caregiver relationship.

Q7: How are caregiver push notifications delivered without the app being open?

Answer: We integrate Google Firebase Cloud Messaging (FCM HTTP v1). When an alert is triggered, Node.js backend pushes a high-priority FCM payload with `priority: 'high'` and native system chime sound directly to the registered caregiver device token.

Q8: What is the time complexity of the 4-factor urgency calculation?

Answer: The algorithm is $O(1)$ constant time. All four factors perform indexed SQLite range queries using indexed timestamp and phone number columns, executing in under 5 milliseconds on modern Android devices.

Q9: How is privacy protected for the elderly person?

Answer: SafeCall performs all raw call analysis and scoring entirely on-device in local SQLite. Audio is never recorded, and only metadata for breakthrough urgency calls (≥ 80) is ever transmitted to the caregiver.

Q10: Why is the threshold set to 80 instead of 50?

Answer: Setting the breakthrough threshold at 80 prevents caregiver alert fatigue. Casual redials or normal daytime calls reach at most 35 to 60 points; only multi-factor emergencies (e.g. repeated calls from emergency contacts at night) reach ≥ 80.
